





QUIZ INSTRUCTION:



Read each question carefully.



Choose only one correct answer.



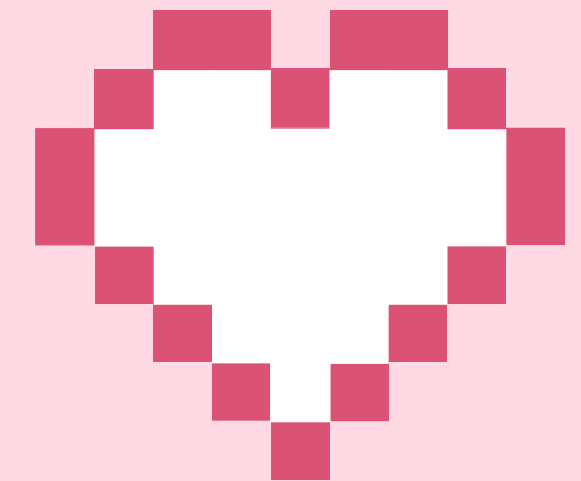
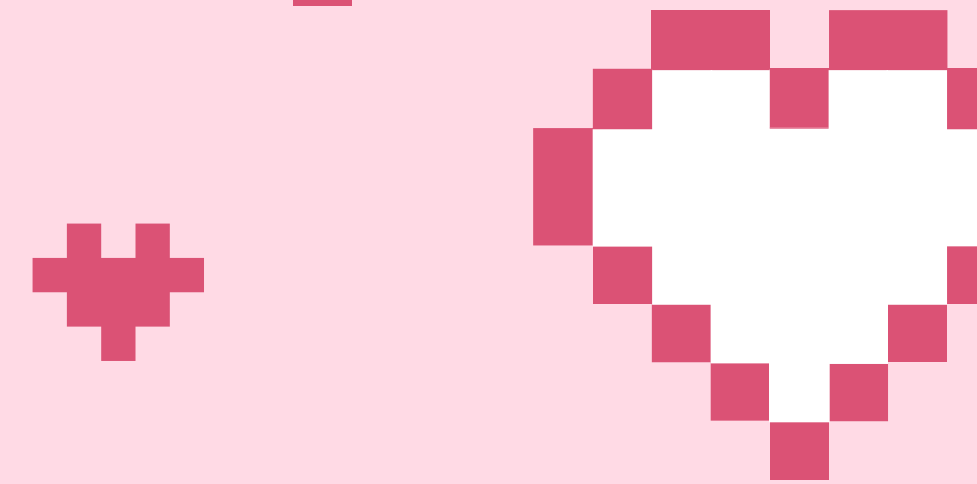
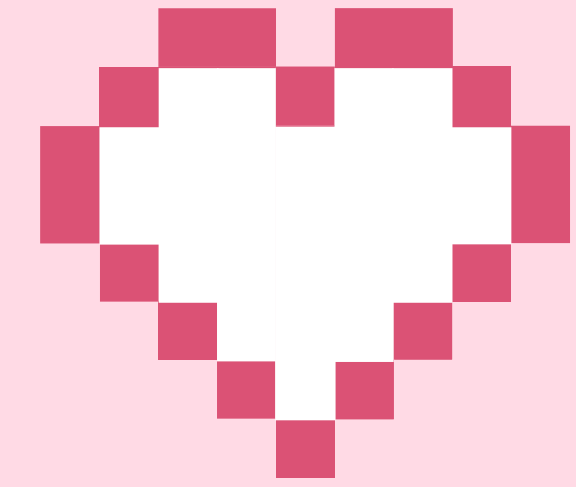
Don't panic if you're unsure – try your best!

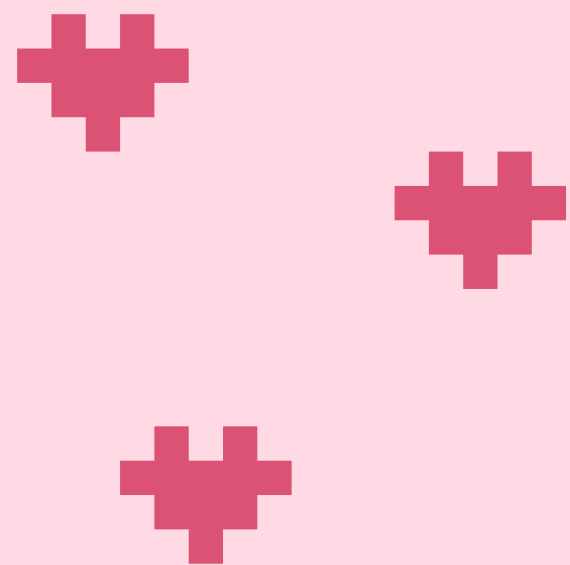
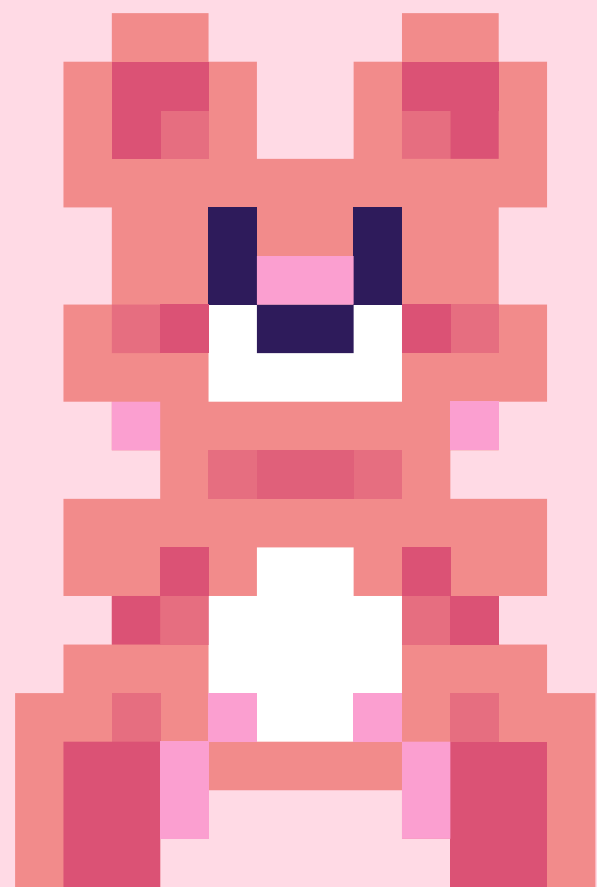


Submit your answer and see your score at the end.



Confirm your answer before proceeding.





-

□

×

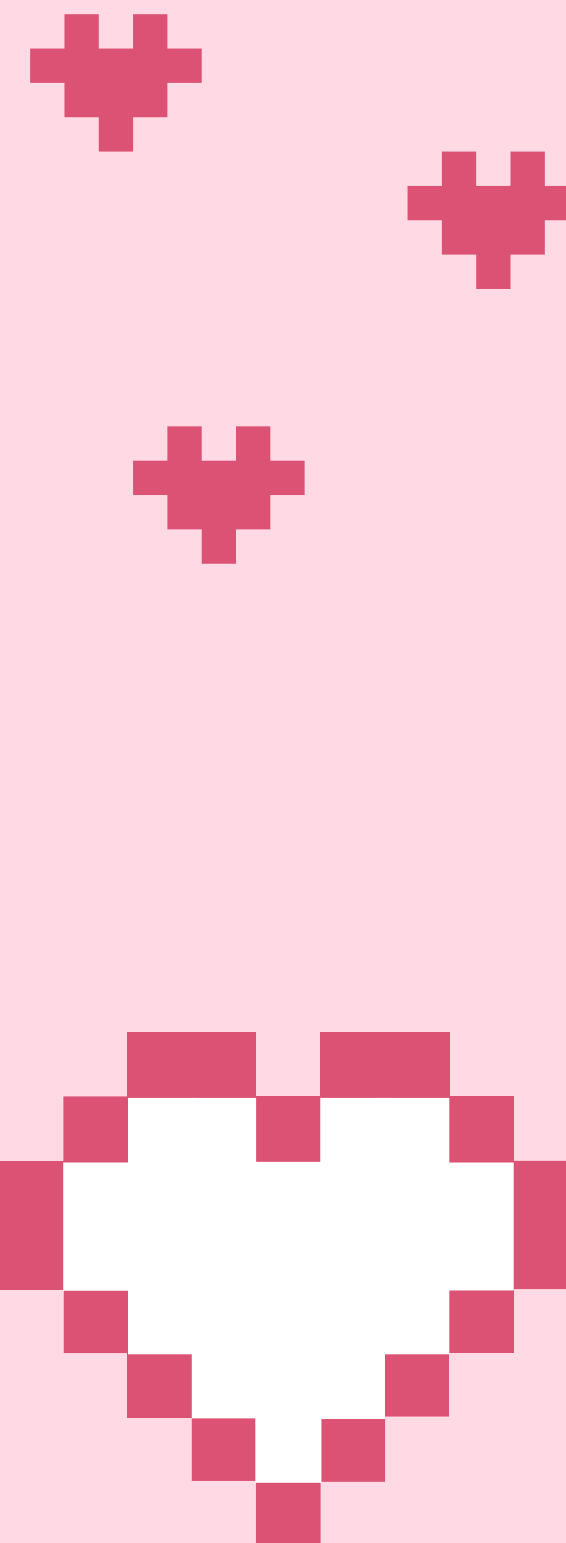
HOW IT WORKS

 JAVASCRIPT LOGIC:

- LOADS QUESTIONS AND DISPLAYS OPTIONS DYNAMICALLY.
- CONFIRMS USER ANSWER BEFORE CHECKING.
- TRACKS SCORE AND DISPLAYS RESULTS AT THE END.

△

▽

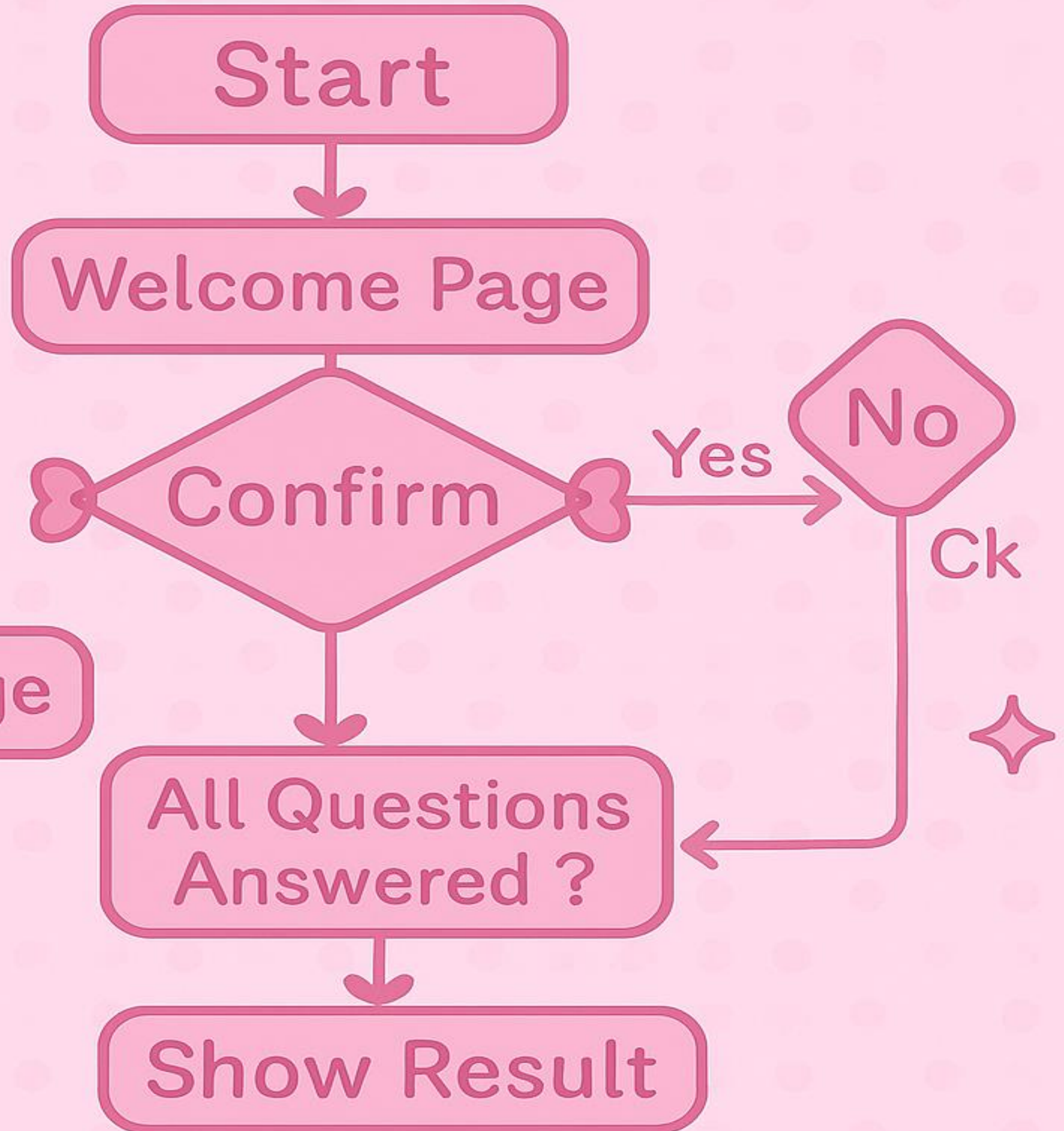


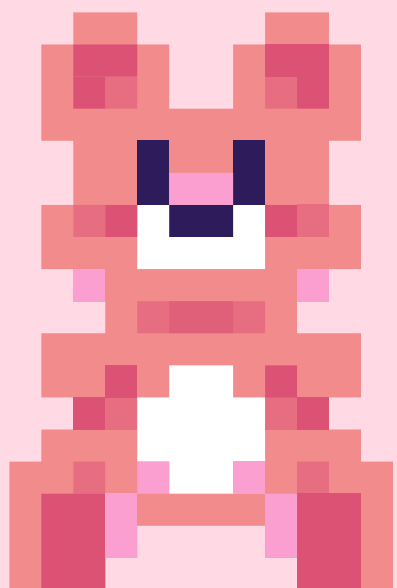
CODE FLOW DIAGRAM



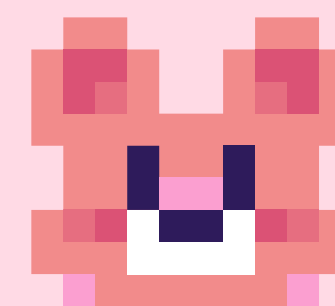
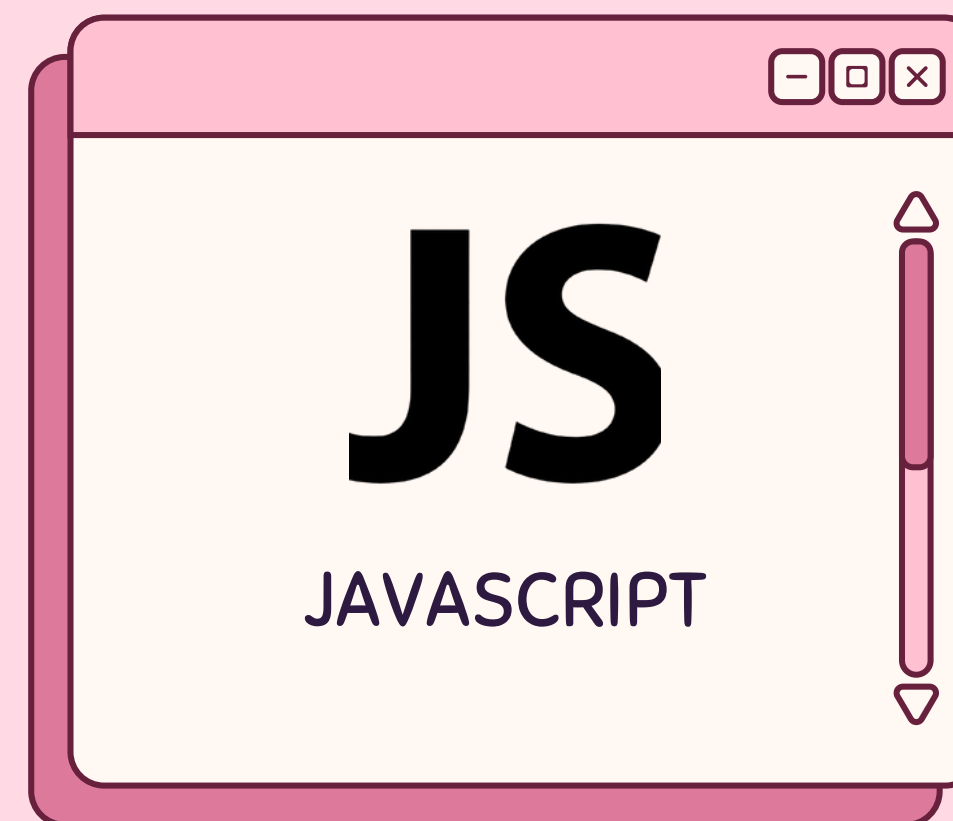
Cute Quiz Time!

Quiz Page





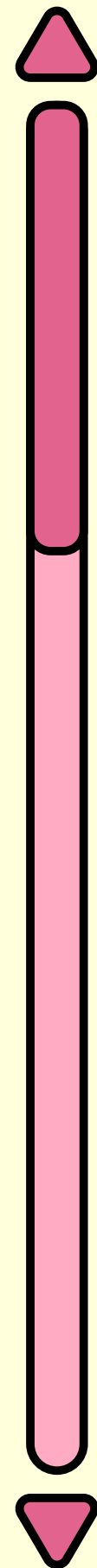
LANGUAGES USED





HTML STRUCTURE

- **Start Screen:** Introduces the quiz, displays lifeline information, and a start button.
- **Quiz Screen:** Displays current questions, multiple-choice options, and lifeline buttons (50/50, Hint, Switch).
- **Result Section:** Shows the final score along with a motivational message.
- External files for CSS (style.css) and JavaScript (script.js) are linked in the `<head>` and just before the `</body>` tag, respectively.





JavaScript STRUCTURE

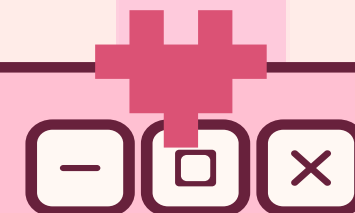
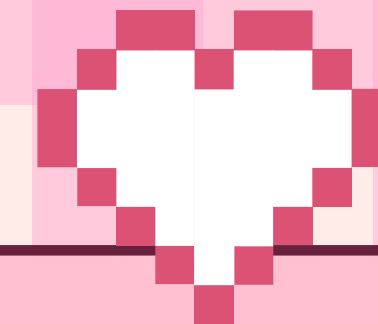
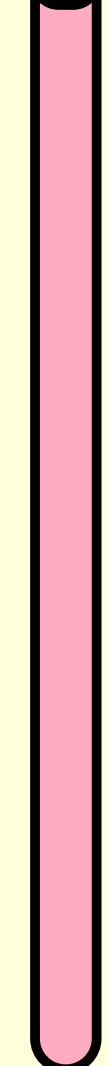
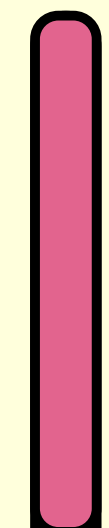
JavaScript Logic Overview

Data Structure

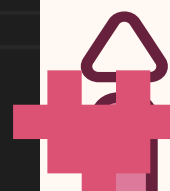
- An array of quiz question objects.
- Each object contains:
 - q: The question
 - options: Four answer choices
 - answer: Correct answer
 - hint: One-liner clue for the question

Core Functionality

- DOM is manipulated dynamically to display content.
- startBtn initializes the quiz.
- loadQuestion() displays the current question and available options.
- confirmAnswer() and checkAnswer() handle answer submission and scoring.



```
33
34 let currentQuestion = 0;
35 let score = 0;
36 let used5050 = false;
37 let usedSwitch = false;
38 const usedHintForQuestion = new Array(quizData.length).fill(false);
39
40 const startContainer = document.getElementById("start-container");
41 const quizContainer = document.getElementById("quiz-container");
42 const questionEl = document.getElementById("question");
43 const optionsEl = document.getElementById("options");
44 const hintTextEl = document.getElementById("hint-text");
45 const resultEl = document.getElementById("result");
46 const btn5050 = document.getElementById("btn-5050");
47 const btnHint = document.getElementById("btn-hint");
48 const btnSwitch = document.getElementById("btn-switch");
49 const submitBtn = document.getElementById("submit-btn");
50 const startBtn = document.getElementById("start-btn");
51
52 startBtn.addEventListener("click", () => {
53   startContainer.style.display = "none";
54   quizContainer.style.display = "block";
55   loadQuestion();
56 });
57
```



CSS STRUCTURE

CSS Styling Highlights

A vibrant gradient background for engaging aesthetics.

Use of **Comic Sans MS** and soft pastel color schemes for a cute and friendly look.

Button hover effects and disabled states enhance interactivity.

Layout is centred using Flexbox and is partially responsive.

```
1 body {  
2   background: linear-gradient(135deg, #ffe0f0, #ffe6b3);  
3   font-family: "Comic Sans MS", "Segoe UI", cursive;  
4   margin: 0;  
5   padding: 0;  
6   display: flex;  
7   justify-content: center;  
8   align-items: center;  
9   height: 100vh;  
10 }  
11  
12 .quiz-container,  
13 .start-container {  
14   background-color: #ffffaf;  
15   border-radius: 16px;  
16   padding: 30px;  
17   box-shadow: 0 0 25px rgba(255, 192, 203, 0.6);  
18   width: 400px;  
19   text-align: center;  
20 }  
21  
22 h1 {  
23   color: #e91e63;  
24   font-size: 28px;  
25   margin-bottom: 20px;  
26 }
```




Future Enhancements



Here are a few possible future improvements:

-  Add Timer: Include a countdown for each question to increase challenge.
-  Question Categories: Let users select from categories like Science, Math, Pop Culture, etc.
-  Scoreboard: Implement a leaderboard to store top scores using localStorage or backend.
-  Multilingual Support: Add the ability to switch between different languages.
-  Dynamic Questions: Load questions from an API or JSON file to make the quiz scalable.
-  Fully Responsive Design: Enhance mobile responsiveness with media queries.
-  Audio/Animations: Add fun sounds and animations to make it more engaging.



Conclusion

The "Quiz Time!" project is a great example of combining design, logic, and creativity. It not only provides an entertaining platform for users but also serves as a hands-on learning opportunity in front-end development. The implementation of lifelines mirrors features from popular quiz shows, making the app more interactive and fun to play. With room for further development, this project can evolve into a more advanced and feature-rich educational game.

